
PRACTICAL – 2

Implementation and Time Analysis of Linear and Binary Search Algorithm

Objective:

The objective of this practical is to implement **Linear Search** and **Binary Search** algorithms and compare their performance based on **time complexity**, **space complexity**, and **execution time**. This helps in understanding the efficiency of both searching techniques and selecting the appropriate algorithm for different applications.

1.Linear Search

Algorithm:

START

INPUT A

INPUT X

FOR $i \leftarrow 0$ TO $\text{len}(A) - 1$

IF $A[i] == X$ THEN

PRINT "Element found at index", i

EXIT

END IF

END FOR

PRINT "Element not found"

STOP

PythonCode:

```
import time
```

```
# Linear Search Function
```

```
def linear_search(arr, n, key):
```

```
    for i in range(n):
```

```
        if arr[i] == key:
```

```
            return i
```

```
    return -1
```

```
# Take input from the user
```

```
n = int(input("Enter the number of elements: "))
```

```
arr = []
```

```
print("Enter the elements:")
```

```
for i in range(n):
```

```
    arr.append(int(input()))
```

```
key = int(input("Enter the element to search: "))
```

```
# Measure execution time
```

```
start_time = time.perf_counter()
```

```
result = linear_search(arr, n, key)
```

```
end_time = time.perf_counter()
# Display result
if result != -1:
    print("\nElement found at position:", result + 1)
else:
    print("\nElement not found")
# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))

#Enter your EN.No
print("EN.No:92460118706")
```

OUTPUT:

```
PS C:\Users\swamy> & 'C:\Users\swamy\AppData\Local\Programs\Python\Python315\python.exe' 'c:\Users\swamy\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58296' '--' 'c:\Users\swamy\import_time.py'
Enter the number of elements: 5
Enter the elements:
6
4
2
8
9
Enter the element to search: 6

Element found at position: 1
Execution Time: 0.0000130000 seconds
EN.No:92460118706
PS C:\Users\swamy> |
```

Time Complexity:

Case	Time Complexity	Description
Best Case	$O(1)$	Element is at the first position.
Average Case	$O(n)$	Element is in the middle or random position.
Worst case	$O(n)$	Element is at the last position or absent.

Space Complexity:

<i>Space Complexity</i>	<i>Description</i>
<i>O(1)</i>	<i>Uses only a few variables; no extra memory is required.</i>

2.Binary Search

Algorithm:

```
START
INPUT sorted list A
INPUT X
low ← 0
high ← len(A)-1
WHILE low <=high
    mid ← (low+high)//2
    IF A[mid]==XTHEN
        PRINT"Elementfoundatindex",mid
        EXIT
    ELSEIFA[mid]<XTHEN
        low←mid+1
    ELSE
        high←mid-1
    END IF
ENDWHILE
PRINT "Element not found"
STOP
```

Python Code

```
import time
# Binary Search Function
def binary_search(arr, n, key):
    low = 0
    high = n - 1
    while low <= high:
```

```
mid = (low + high) // 2
if arr[mid] == key:
    return mid
elif arr[mid] < key:
    low = mid + 1
else:
    high = mid - 1
return -1
# Take input from the user
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key = int(input("Enter the element to search: "))
# Sort the array
arr.sort()
# Measure execution time
start_time = time.perf_counter()
result = binary_search(arr, n, key)
end_time = time.perf_counter()
# Display sorted array
print("\nSorted Array:", arr)
# Display result
if result != -1:
    print("Element found at position:", result + 1)
else:
    print("Element not found")

# Display execution time
execution_time = end_time - start_time
print("Execution Time: {:.10f} seconds".format(execution_time))

#Enter your EN.No
print("EN.No:92460118706")
```

OUTPUT:

```
PS C:\Users\swamy> c:: cd 'c:\Users\swamy' ; & 'c:\Users\swamy\AppData\Local\Programs\Python\Python315\python.exe'  
'c:\Users\swamy\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '64723' '--'  
'c:\Users\swamy\import time.py'  
Enter the number of elements: 5  
Enter the elements:  
8  
9  
3  
1  
5  
Enter the element to search: 1  
  
Sorted Array: [1, 3, 5, 8, 9]  
Element found at position: 1  
Execution Time: 0.0000121000 seconds  
EN.No:92460118706  
PS C:\Users\swamy>
```

Time Complexity:

Case	Time Complexity	Description
Best Case	$O(1)$	Element is found at the middle.
Average Case	$O(\log n)$	Search space is halved repeatedly.
Worst case	$O(\log n)$	Maximum divisions before finding or rejecting the element.

Space Complexity:

Space Complexity	Description
$O(1)$	Uses only a few variables (low, high, mid); no extra memory is required.